

Altec Website Rebuild

Architecture & Content Management Proposal

Prepared 2026-07-14 · Revised 2026-07-16 (stack pivot — see below) · Status: implemented and running locally

REVISION NOTE (2026-07-16)

The originally proposed stack (Next.js + Node.js + Prisma) was replaced with **PHP + React** after confirming the target GoDaddy shared cPanel hosting has no Node.js application support — only PHP, Python, and Ruby app selectors are available. Sections 4, 5, 9, and 10 below reflect the current, implemented architecture. Everything else in this document (business requirements, IA, data model) is unchanged.

Contents

1. Project Summary
2. Business Requirements Recap
3. Design Reference — What We're Borrowing From Viessmann.it
4. Technology Stack
5. System Architecture
6. Data Model
7. Admin CMS — Feature List
8. Public Site — Proposed Page Structure
9. Hosting & Deployment
10. Security — Current State
11. Suggested Next Steps

1. Project Summary

Altec's current website (altec-al.com) is a WordPress site. The goal is to rebuild it as a modern, self-managed website — without WordPress — that the owner can update himself as new products and projects come in, running entirely on the existing shared cPanel hosting with no external/cloud services.

2. Business Requirements Recap

- **Core business:** AC (air conditioning) sales and installation is the owner's primary activity. Reconstruction and home furnishing ("arredamento") are secondary, occasional services — the new site's structure reflects that weighting, unlike the current site which gives all services/products roughly equal billing.
- **Partnership:** the owner collaborates with **Viessmann** (heating/AC/energy systems manufacturer) — this relationship is visible on the site (branding, product lines carried).
- **Ongoing content growth:** products, gallery items, and news are added continuously — this is why a real CMS was built rather than a static, hand-coded site.
- **Multi-language:** current site supports English/Albanian; the new site keeps at least these two languages, managed from the CMS.
- **No e-commerce:** products are informational/catalog-style, not a shopping cart. A working contact/quote-request form was added — the old site conspicuously lacked one.
- **Hosting constraint:** must run entirely on the existing shared cPanel server. No external databases, cloud CMS, or third-party hosted services.

3. Design Reference — What We're Borrowing From Viessmann.it

The owner specifically likes viessmann.it's structure. At Altec's scale, the useful patterns are:

- **Hero + promo sections** on the homepage (current offers, featured products/brands)
- **Card-based product categories** rather than long text lists
- **Solution-based grouping** — organize by use-case (e.g. "For Homes" / "For Offices & Commercial") in addition to flat product categories
- **References/case studies gallery**, segmented by project type (matches the existing "Works" portfolio concept)
- **A news/blog section** for updates and completed projects

Deliberately **not** replicating (too large-scale/B2B2C for a single local installer): partner-installer locator portal, financing simulators, training academy, restricted dealer portal.

4. Technology Stack Revised

Layer	Choice	Notes
Public site	Plain PHP	Server-rendered per request, no build step, no JS framework. Runs natively on any PHP host — including this one. Chosen after confirming the hosting has no Node.js support.

Admin / CMS panel	React + TypeScript, built with Vite	Compiled locally to static HTML/CSS/JS, deployed under <code>/admin/</code> . No Node process on the server — Node is a local build tool only, exactly like a past React project the owner already deployed successfully on this same host.
API	Plain PHP REST endpoints	Single front controller (<code>/api/index.php</code>) dispatches to per-resource handler files. Used by the admin panel only; the public site queries the database directly.
Auth	PHP native sessions	<code>password_hash()</code> / <code>password_verify()</code> (bcrypt), httpOnly session cookie. No external auth library needed.
Database	MySQL / MariaDB	Accessed via PDO with prepared statements, no ORM. Bundled free with the cPanel hosting account.
Image/file storage	Local disk on the cPanel server	Uploaded via the admin panel, served directly — no external CDN/storage service.
Hosting	Existing GoDaddy shared cPanel account	Confirmed via cPanel screenshot: PHP (with version selector), Python, and Ruby app support present; no Node.js application support. This is what drove the stack choice above.

5. System Architecture



The admin panel is the only piece of React in this system, and it never runs on the server as a live process — it's compiled ahead of time (`npm run build`) into plain HTML/CSS/JS and uploaded like any other static asset. It talks to the same PHP API the rest of the system uses, over ordinary `fetch()` calls with the session cookie included.

6. Data Model

Core entities the CMS manages. Field lists are indicative, not exhaustive.

Entity	Key Fields	Notes
Page	slug, title (per language), body, SEO meta	About Us, Partners, etc. — flexible content pages
MenuItem	label (per language), link type/target, parent, sort order, visible	Self-referencing for nested menus, owner-editable order

Category	name (per language), type (service / product), parent	Powers "Services" and "Products" groupings
Product	name, description (per language), category, brand (e.g. Viessmann), images, published	
WorkItem	title, project type, images, description, date	Portfolio / "Works" section, filterable by project type
NewsPost	slug, title, body, cover image, published date	Blog/news/updates section
Setting	key, value	Key-value store for contact info, social links, homepage promo text, etc.
ContactSubmission	name, email, phone, message, created at	Stores inquiries from the contact form
AdminUser	email, password hash, role	Starts with a single admin (the owner)
Language	code (en/al), label	Drives which translations are shown/editable

7. Admin CMS — Feature List

- Secure login (single admin account)
- **Products & Categories:** create/edit/delete, upload photos, mark published/unpublished, per-language text
- **Menu builder:** add/remove/reorder menu items, nest submenus, point items at pages, categories, or custom URLs
- **Works/Gallery:** add projects with photos, tag by project type
- **News:** write/publish/edit posts
- **Settings:** contact details, social links, homepage promotional content
- **Contact form submissions:** inbox view of inquiries received through the site
- Per-language editing (English / Albanian) for every content type

8. Public Site — Page Structure

1. **Home** — hero/promo, featured AC products & Viessmann partnership, recent projects, recent news
2. **AC Sales & Installation** (*primary section*) — products by category, brands carried
3. **Reconstruction & Home Furnishing** (*secondary section*)
4. **About Us**

5. **Partners** (Viessmann and others)
6. **Works / Gallery** — project portfolio
7. **News**
8. **Contact** — contact form + office details

Every page above is driven by the menu builder and content entities from Section 6, so the owner can rename, reorder, add, or remove sections without a code change.

9. Hosting & Deployment

CONFIRMED WORKING

Node.js is not required anywhere on the server. Every component below runs as either plain PHP (executed per-request by the existing web server) or static files — both are things this GoDaddy shared cPanel account already supports today, no plan upgrade or provider change needed.

- Upload the contents of the app's web root as-is — PHP runs automatically, no app manager/setup step required
- MySQL database created through cPanel's MySQL Database Wizard (already standard on this account)
- Database schema created by importing a plain SQL file; a seed script populates default admin/settings/menu data
- The admin panel is built locally (`npm run build`) and its output uploaded once per change — no ongoing build process on the server
- Uploaded images stored under the web root, served directly by the existing web server
- SSL/TLS via cPanel's existing certificate tools

10. Security — Current State

What's implemented:

- Passwords hashed with bcrypt; prepared statements throughout (no SQL injection); output escaped everywhere (no unescaped user input rendered as HTML)
- Admin session cookie is httpOnly and SameSite=Lax; every admin-only API endpoint checks the session server-side
- Database credentials file lives outside the web root — not reachable by direct URL

Not yet done — recommended before handling real public traffic:

- CSRF tokens on admin state-changing requests (currently relying on SameSite alone)
- Login rate-limiting / logout
- Spam protection (e.g. honeypot or CAPTCHA) on the public contact form

- Security headers (CSP, X-Frame-Options, etc.)
- Enforcing HTTPS site-wide at the hosting level

11. Suggested Next Steps

1. Visual design pass on the public site (currently functional but unstyled — this was deliberately deferred until the stack was settled)
2. Address the security items in Section 10 before launch
3. Deploy to the production GoDaddy account and point the domain at it
4. Replace placeholder/test content with real products, projects, and copy